

Woxsen University

School of Technology



PROJECT REPORT

on

ACADEMIC CONFERENCE MANAGEMENT SYSTEM

Design and Implementation of a DBMS for Academic Conferences

Name: Shriansh Singh

Roll Number: 25WU0102259

Course: Database Management Systems (DBMS)

Faculty Name: Dr. Kiran Mayee

Academic Year: 2026–27

ABSTRACT

Managing an academic conference involves many connected activities: collecting paper submissions, tracking authors, assigning reviewers, recording scores, taking acceptance decisions and informing authors. When these are handled through emails and spreadsheets, data becomes duplicated, inconsistent and hard to track. This project presents the design and implementation of a relational database management system for academic conference paper submission and review.

The database, named `conferencedb`, is implemented in MySQL and consists of ten tables: `ROLE`, `USER_ACCOUNT`, `AUTHOR`, `REVIEWER`, `CONFERENCE`, `PAPER`, `PAPER_AUTHOR`, `REVIEW`, `DECISION` and `NOTIFICATION`. The design is based on an entity-relationship model and is normalised to the third normal form. Primary keys, foreign keys, `UNIQUE` and `NOT NULL` constraints enforce integrity, a junction table resolves the many-to-many relationship between papers and authors, and a role table supports role-based access for administrators, authors and reviewers.

The report documents the objectives, requirements, ER diagram, relational schema, normalisation, data dictionary, DDL and DML commands, analytical queries with outputs, user interface design, implementation details and testing. Constraint tests confirmed that duplicate e-mails, invalid foreign keys, duplicate decisions, null passwords and orphan deletions are all rejected by the database. The resulting system provides a centralised, transparent and scalable foundation for managing academic conferences.

Keywords: DBMS, MySQL, Entity-Relationship Model, Normalization, SQL, Conference Management, Peer Review, Role-Based Access Control

TABLE OF CONTENTS

1. INTRODUCTION AND PROBLEM STATEMENT	1
1.1 Introduction.....	1
1.2 Problem Statement.....	1
2. OBJECTIVES AND SCOPE	1
2.1 Objectives	1
2.2 Scope.....	2
2.3 Limitations	2
3. SOFTWARE AND HARDWARE REQUIREMENTS.....	2
3.1 Software Requirements	2
3.2 Hardware Requirements.....	2
4. ER DIAGRAM.....	4
4.1 Entities and Relationships.....	4
5. RELATIONAL SCHEMA AND NORMALIZATION.....	5
5.1 Relational Schema	5
5.2 Normalization	6
5.2.1 First Normal Form (1NF)	6
5.2.2 Second Normal Form (2NF).....	6
5.2.3 Third Normal Form (3NF).....	6
5.2.4 Boyce-Codd Normal Form (BCNF).....	7
6. DATA DICTIONARY	8
7. SQL COMMANDS USED (DDL AND DML) WITH SAMPLE OUTPUTS	12
7.1 Data Definition Language (DDL).....	12
7.2 Data Manipulation Language (DML)	13
7.3 Sample Outputs	14
8. QUERIES WITH OUTPUTS	16
Query 1: Papers with their conference and status.....	16
Query 2: Authors of each paper (many-to-many join)	16
Query 3: Average review score per paper	16
Query 4: Number of reviews done by each reviewer	17
Query 5: Number of decisions by outcome	17
Query 6: Papers still waiting for a decision.....	17
Query 7: Acceptance rate.....	18
Query 8: Notifications sent to authors	18
Query 9: Query given during Presentation-II	18

9. UI DESIGN AND SCREENSHOTS	19
9.1 Login Screen.....	19
9.2 Admin Dashboard.....	19
9.3 Author Paper Submission and Tracking	19
9.4 Reviewer Panel	19
10. IMPLEMENTATION DETAILS.....	20
10.1 Technology Stack.....	20
10.2 Database Connectivity	20
10.3 Key Code Snippets.....	20
10.4 Integrity and Access Control	21
11. TESTING.....	22
11.1 Test Cases and Results.....	22
11.2 Observations	23
12. CONCLUSION AND FUTURE ENHANCEMENTS	24
12.1 Conclusion	24
12.2 Future Enhancements.....	24
13. REFERENCES.....	24
APPENDIX: GITHUB REPOSITORY LINK.....	25

1. INTRODUCTION AND PROBLEM STATEMENT

1.1 Introduction

Academic conferences are the main venue for presenting and discussing research. Behind every conference is a workflow in which authors submit papers, programme committee members review them, and organisers decide which papers to accept. The volume of data in this workflow, which includes author details, paper metadata, review scores, comments, decisions and notifications, grows quickly with the size of the conference.

A database management system provides a reliable way to store this data once and use it consistently [1]. This project builds a relational database for academic conference management that follows the standard design process of conceptual modelling with an ER diagram [5], conversion to relational schemas [4], normalisation and implementation in SQL.

1.2 Problem Statement

Conference organisers who rely on e-mail threads and spreadsheets face several problems:

- Paper and author information is scattered and duplicated across files, which leads to inconsistencies
- There is no structured way to assign reviewers or to see how many papers each reviewer is handling
- Review scores, comments and final decisions are hard to connect to the right paper
- Authors have no real-time view of the status of their submissions
- There is no access control, so sensitive information such as reviewer comments can be seen by the wrong people

The problem addressed is the design and implementation of a centralised relational database that stores all conference data with integrity, supports the complete submission-to-decision workflow and can answer management queries on demand.

2. OBJECTIVES AND SCOPE

2.1 Objectives

1. To provide a single unified platform for paper submission and conference management.
2. To allow seamless paper submission with metadata validation and author tracking.

3. To support a structured peer-review workflow of reviewer assignment, scoring and comments.
4. To record decisions (Accept, Reject or Revise) and notify authors of outcomes.
5. To implement role-based access for administrators, authors and reviewers.
6. To guarantee data integrity through keys, constraints and normalisation.
7. To generate submission statistics, review progress and acceptance rates on demand.

2.2 Scope

The scope covers the database design and implementation for one or more conferences, including authors, co-authored papers, reviewers, reviews, decisions, user accounts with roles, and notifications. SQL queries produce the required reports. The user interface layer is described in Chapter 9.

2.3 Limitations

- The system manages metadata; storing the full paper file is outside the current schema
- Each paper currently has at most one decision, and review rounds are not tracked separately
- Reviewer assignment is recorded through the REVIEW table and is not yet automated

3. SOFTWARE AND HARDWARE REQUIREMENTS

3.1 Software Requirements

Component	Requirement
Database management system	MySQL (database: conferencedb)
Database client	MySQL command-line client
Operating system	[Windows 10/11 / Linux / macOS]
Front end / back end	[Technology used for the application layer]
ER diagram tool	[Tool used to draw the ER diagram]
Version control	Git and GitHub

Table 3.1: Software Requirements

3.2 Hardware Requirements

Component	Minimum requirement
Processor	Dual-core processor, 2 GHz or higher
Memory (RAM)	4 GB (8 GB recommended)
Storage	500 MB free disk space
Display	1366 × 768 resolution or higher

Table 3.2: Hardware Requirements

4. ER DIAGRAM

The conceptual design is shown in Figure 4.1. It contains nine entities and one junction entity, with relationships named in the diamonds. PK denotes a primary key and FK a foreign key.

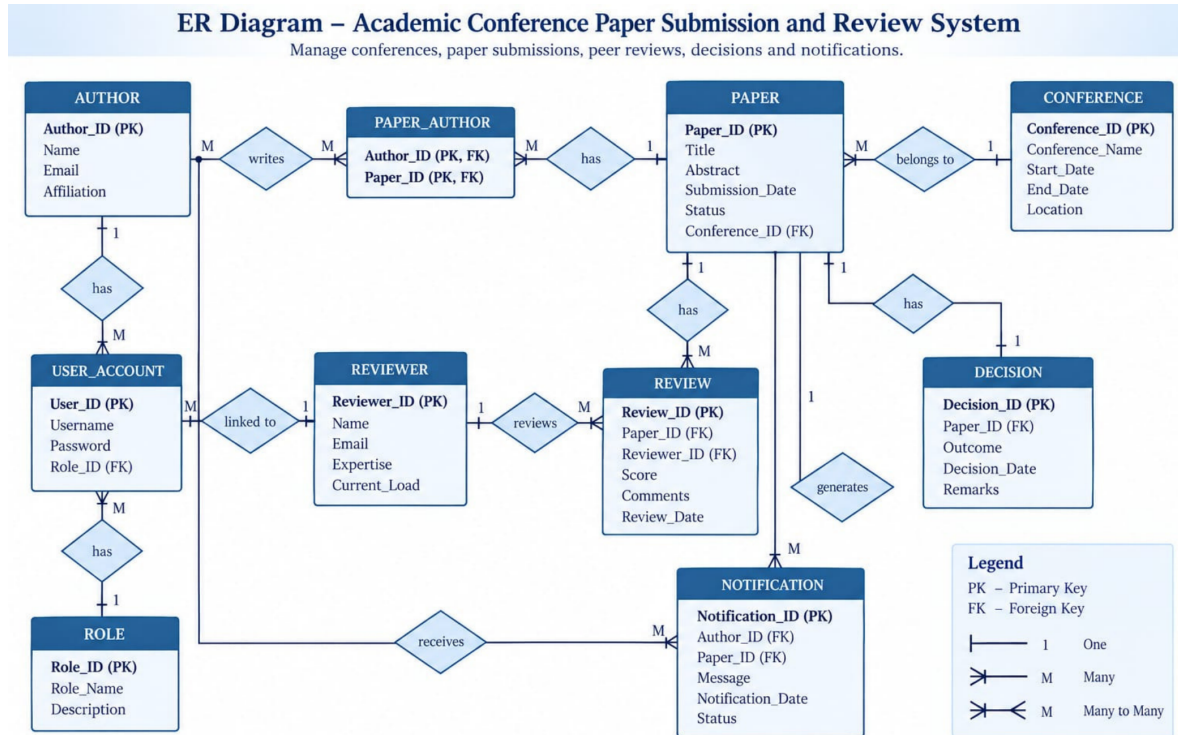


Figure 4.1: ER Diagram – Academic Conference Paper Submission and Review System

4.1 Entities and Relationships

- **AUTHOR – PAPER:** An author writes many papers and a paper can have many authors (M:N), resolved by the PAPER_AUTHOR junction entity.
- **CONFERENCE – PAPER:** A conference has many papers, and each paper belongs to one conference (1:M).
- **PAPER – REVIEW – REVIEWER:** A paper receives many reviews, and each review is written by one reviewer (1:M on both sides).
- **PAPER – DECISION:** A paper has one final decision (1:1).
- **PAPER – NOTIFICATION – AUTHOR:** A paper generates notifications that are received by authors (1:M).
- **ROLE – USER_ACCOUNT:** A role is assigned to many user accounts (1:M), and user accounts are linked to authors and reviewers.

5. RELATIONAL SCHEMA AND NORMALIZATION

5.1 Relational Schema

The ER diagram was converted into the relations in Table 5.1. Primary keys are marked (PK) and foreign keys (FK).

Relation	Attributes	Foreign keys
ROLE	Role_ID (PK), Role_Name, Description	–
AUTHOR	Author_ID (PK), Name, Email, Affiliation	–
CONFERENCE	Conference_ID (PK), Conference_Name, Start_Date, End_Date, Location	–
REVIEWER	Reviewer_ID (PK), Name, Email, Expertise, Current_Load	–
USER_ACCOUNT	User_ID (PK), Username, Password, Role_ID (FK)	Role_ID → ROLE
PAPER	Paper_ID (PK), Title, Abstract, Submission_Date, Status, Conference_ID (FK)	Conference_ID → CONFERENCE
PAPER_AUTHOR	Paper_ID (PK, FK), Author_ID (PK, FK)	Paper_ID → PAPER; Author_ID → AUTHOR
REVIEW	Review_ID (PK), Paper_ID (FK), Reviewer_ID (FK), Score, Comments, Review_Date	Paper_ID → PAPER; Reviewer_ID → REVIEWER
DECISION	Decision_ID (PK), Paper_ID (FK, UNIQUE), Outcome, Decision_Date, Remarks	Paper_ID → PAPER
NOTIFICATION	Notification_ID (PK), Author_ID (FK), Paper_ID (FK), Message, Notification_Date, Status	Author_ID → AUTHOR; Paper_ID → PAPER

Table 5.1: Relational Schema

Table Schemas with Primary and Foreign Keys	
Academic Conference Paper Submission and Review System	
	DESIGN MANAGE REVIEW DECIDE
Table	Key Columns & Constraints
ROLE	Role_ID PK , Role_Name NOT NULL, Description
AUTHOR	Author_ID PK , Name NOT NULL, Email NOT NULL UNIQUE, Affiliation
CONFERENCE	Conference_ID PK , Conference_Name NOT NULL, Start_Date, End_Date, Location
REVIEWER	Reviewer_ID PK , Name NOT NULL, Email NOT NULL UNIQUE, Expertise, Current_Load
USER_ACCOUNT	User_ID PK , Username NOT NULL UNIQUE, Password NOT NULL, Role_ID FK
PAPER	Paper_ID PK , Title NOT NULL, Abstract, Submission_Date, Status, Conference_ID FK
PAPER_AUTHOR	Paper_ID PK FK , Author_ID PK FK
REVIEW	Review_ID PK , Paper_ID FK , Reviewer_ID FK , Score, Comments, Review_Date
DECISION	Decision_ID PK , Paper_ID FK UNIQUE, Outcome, Decision_Date, Remarks
NOTIFICATION	Notification_ID PK , Author_ID FK , Paper_ID FK , Message, Notification_Date, Status

PK = Primary Key
(Uniquely identifies a record)

FK = Foreign Key
(References a primary key in another table)

RESEARCH TODAY
A BETTER TOMORROW

Figure 5.1: Table Schemas with Primary and Foreign Keys

5.2 Normalization

Normalisation removes redundancy and update anomalies by organising attributes into well-structured relations [4], [5]. The design was checked against the first three normal forms and Boyce-Codd normal form.

5.2.1 First Normal Form (1NF)

All attributes hold single, atomic values and every relation has a primary key. A paper can have several authors, so storing authors as a list inside PAPER would violate 1NF; instead each (paper, author) pair is stored as one row in PAPER_AUTHOR.

5.2.2 Second Normal Form (2NF)

Every relation is in 1NF and no non-key attribute depends on only part of a composite key. All relations except PAPER_AUTHOR have a single-column primary key, so partial dependency cannot occur. PAPER_AUTHOR has only key attributes, so it also satisfies 2NF.

5.2.3 Third Normal Form (3NF)

No non-key attribute depends on another non-key attribute. Examples: conference name, dates and location are stored in CONFERENCE and referenced from PAPER by Conference_ID, not repeated for every paper; role names are stored in ROLE and referenced from

USER_ACCOUNT; reviewer name and expertise are stored in REVIEWER and referenced from REVIEW.

5.2.4 Boyce-Codd Normal Form (BCNF)

In every relation each determinant is a candidate key. In AUTHOR, REVIEWER and USER_ACCOUNT, Email and Username are alternate candidate keys enforced with UNIQUE. In DECISION, Paper_ID is also a candidate key because each paper has only one decision.

Design note: Current_Load in REVIEWER can be derived from the number of reviews assigned. It is kept as a stored attribute for fast workload checks, which is a deliberate trade-off between strict normalisation and query speed.

6. DATA DICTIONARY

The data dictionary describes the structure of every table in conferencedb: column names, data types, constraints and meaning.

Column	Data Type	Constraints	Description
Role_ID	INT	PRIMARY KEY	Unique role identifier
Role_Name	VARCHAR(30)	NOT NULL	Name of the role (Admin, Author, Reviewer)
Description	VARCHAR(100)	–	What the role is allowed to do

Table 6.1: ROLE

Column	Data Type	Constraints	Description
Author_ID	INT	PRIMARY KEY	Unique author identifier
Name	VARCHAR(100)	NOT NULL	Full name of the author
Email	VARCHAR(100)	NOT NULL, UNIQUE	Contact e-mail address
Affiliation	VARCHAR(150)	–	University or organisation

Table 6.2: AUTHOR

Column	Data Type	Constraints	Description
Conference_ID	INT	PRIMARY KEY	Unique conference identifier
Conference_Name	VARCHAR(150)	NOT NULL	Title of the conference
Start_Date	DATE	–	First day of the conference
End_Date	DATE	–	Last day of the conference
Location	VARCHAR(100)	–	City or venue

Table 6.3: CONFERENCE

Column	Data Type	Constraints	Description
Reviewer_ID	INT	PRIMARY KEY	Unique reviewer identifier
Name	VARCHAR(100)	NOT NULL	Name of the reviewer
Email	VARCHAR(100)	NOT NULL, UNIQUE	Contact e-mail address
Expertise	VARCHAR(100)	–	Area of expertise
Current_Load	INT	DEFAULT 0	Number of papers currently assigned

Table 6.4: REVIEWER

Column	Data Type	Constraints	Description
User_ID	INT	PRIMARY KEY	Unique account identifier
Username	VARCHAR(50)	NOT NULL, UNIQUE	Login name
Password	VARCHAR(100)	NOT NULL	Login password
Role_ID	INT	FOREIGN KEY → ROLE	Role of the account holder

Table 6.5: USER_ACCOUNT

Column	Data Type	Constraints	Description
Paper_ID	INT	PRIMARY KEY	Unique paper identifier
Title	VARCHAR(200)	NOT NULL	Title of the paper
Abstract	TEXT	–	Summary of the paper
Submission_Date	DATE	–	Date of submission
Status	VARCHAR(30)	–	Under Review, Accepted, Revision, etc.
Conference_ID	INT	FOREIGN KEY → CONFERENCE	Conference the paper is submitted to

Table 6.6: PAPER

Column	Data Type	Constraints	Description
Paper_ID	INT	PRIMARY KEY (composite), FOREIGN KEY → PAPER	Paper written
Author_ID	INT	PRIMARY KEY (composite), FOREIGN KEY → AUTHOR	Author of the paper

Table 6.7: PAPER_AUTHOR

Column	Data Type	Constraints	Description
Review_ID	INT	PRIMARY KEY	Unique review identifier
Paper_ID	INT	FOREIGN KEY → PAPER	Paper being reviewed
Reviewer_ID	INT	FOREIGN KEY → REVIEWER	Reviewer who wrote the review
Score	INT	–	Score given by the reviewer
Comments	VARCHAR(255)	–	Reviewer comments
Review_Date	DATE	–	Date of the review

Table 6.8: REVIEW

Column	Data Type	Constraints	Description
Decision_ID	INT	PRIMARY KEY	Unique decision identifier
Paper_ID	INT	FOREIGN KEY → PAPER, UNIQUE	Paper the decision is for
Outcome	VARCHAR(20)	–	Accept, Reject or Revise
Decision_Date	DATE	–	Date of the decision
Remarks	VARCHAR(255)	–	Reason for the decision

Table 6.9: DECISION

Column	Data Type	Constraints	Description
Notification_ID	INT	PRIMARY KEY	Unique notification identifier
Author_ID	INT	FOREIGN KEY → AUTHOR	Author who receives the message
Paper_ID	INT	FOREIGN KEY → PAPER	Paper the message is about
Message	VARCHAR(255)	–	Notification text
Notification_Date	DATE	–	Date sent
Status	VARCHAR(20)	–	Delivery status, for example Sent

Table 6.10: NOTIFICATION

7. SQL COMMANDS USED (DDL AND DML) WITH SAMPLE OUTPUTS

7.1 Data Definition Language (DDL)

The database and its ten tables were created with CREATE DATABASE and CREATE TABLE statements. Parent tables are created before the child tables that reference them.

```
CREATE DATABASE conferencedb;
USE conferencedb;
CREATE TABLE ROLE (
    Role_ID INT PRIMARY KEY,
    Role_Name VARCHAR(30) NOT NULL,
    Description VARCHAR(100)
);
CREATE TABLE AUTHOR (
    Author_ID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Affiliation VARCHAR(150)
);
CREATE TABLE CONFERENCE (
    Conference_ID INT PRIMARY KEY,
    Conference_Name VARCHAR(150) NOT NULL,
    Start_Date DATE,
    End_Date DATE,
    Location VARCHAR(100)
);
CREATE TABLE REVIEWER (
    Reviewer_ID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Expertise VARCHAR(100),
    Current_Load INT DEFAULT 0
);
```

Listing 7.1: DDL for the database and independent tables

```
CREATE TABLE USER_ACCOUNT (
    User_ID INT PRIMARY KEY,
    Username VARCHAR(50) NOT NULL UNIQUE,
    Password VARCHAR(100) NOT NULL,
    Role_ID INT,
    FOREIGN KEY (Role_ID) REFERENCES ROLE(Role_ID)
);
CREATE TABLE PAPER (
    Paper_ID INT PRIMARY KEY,
    Title VARCHAR(200) NOT NULL,
    Abstract TEXT,
    Submission_Date DATE,
    Status VARCHAR(30),
    Conference_ID INT,
    FOREIGN KEY (Conference_ID) REFERENCES CONFERENCE(Conference_ID)
);
CREATE TABLE PAPER_AUTHOR (
    Paper_ID INT,
    Author_ID INT,
    PRIMARY KEY (Paper_ID, Author_ID),
    FOREIGN KEY (Paper_ID) REFERENCES PAPER(Paper_ID),
    FOREIGN KEY (Author_ID) REFERENCES AUTHOR(Author_ID)
);
CREATE TABLE REVIEW (
```

```

Review_ID INT PRIMARY KEY,
Paper_ID INT,
Reviewer_ID INT,
Score INT,
Comments VARCHAR(255),
Review_Date DATE,
FOREIGN KEY (Paper_ID) REFERENCES PAPER(Paper_ID),
FOREIGN KEY (Reviewer_ID) REFERENCES REVIEWER(Reviewer_ID)
);
CREATE TABLE DECISION (
    Decision_ID INT PRIMARY KEY,
    Paper_ID INT UNIQUE,
    Outcome VARCHAR(20),
    Decision_Date DATE,
    Remarks VARCHAR(255),
    FOREIGN KEY (Paper_ID) REFERENCES PAPER(Paper_ID)
);
CREATE TABLE NOTIFICATION (
    Notification_ID INT PRIMARY KEY,
    Author_ID INT,
    Paper_ID INT,
    Message VARCHAR(255),
    Notification_Date DATE,
    Status VARCHAR(20),
    FOREIGN KEY (Author_ID) REFERENCES AUTHOR(Author_ID),
    FOREIGN KEY (Paper_ID) REFERENCES PAPER(Paper_ID)
);

```

Listing 7.2: DDL for tables with foreign keys

7.2 Data Manipulation Language (DML)

Sample data was inserted with INSERT statements, again in parent-to-child order.

```

INSERT INTO ROLE VALUES (1,'Admin','Manages conference and system'),
(2,'Author','Submits and tracks papers'),
(3,'Reviewer','Reviews submitted papers');
INSERT INTO AUTHOR VALUES (101,'Rahul Sharma','rahul@email.com','Woxsen
University'),
(102,'Priya Reddy','priya@email.com','IIT Hyderabad'),
(103,'Arjun Kumar','arjun@email.com','Osmania University');
INSERT INTO CONFERENCE VALUES (1,'Woxsen AI Conference 2026','2026-10-20','2026-
10-22','Hyderabad');
INSERT INTO REVIEWER VALUES (201,'Dr. Anil Mehta','anil@reviewer.com','Artificial
Intelligence',2),
(202,'Dr. Neha Rao','neha@reviewer.com','Machine Learning',1);
INSERT INTO USER_ACCOUNT VALUES
(301,'admin01','admin123',1),(302,'rahul01','rahul123',2),
(303,'priya01','priya123',2),(304,'anil01','anil123',3);
INSERT INTO PAPER VALUES
(1001,'AI in Healthcare','Application of AI in modern healthcare systems','2026-
09-10','Under Review',1),
(1002,'Secure ML Systems','Security techniques for machine learning
systems','2026-09-11','Accepted',1),
(1003,'NLP for Education','Using NLP to improve educational systems','2026-09-
12','Revision',1);
INSERT INTO PAPER_AUTHOR VALUES (1001,101),(1001,102),(1002,102),(1003,103);
INSERT INTO REVIEW VALUES
(501,1001,201,8,'Strong methodology and good implementation.','2026-09-13'),
(502,1002,202,9,'Excellent research work.','2026-09-14'),
(503,1003,201,6,'The paper needs improvement in methodology.','2026-09-14');
INSERT INTO DECISION VALUES
(601,1002,'Accept','2026-09-15','Paper meets conference requirements.'),
(602,1003,'Revise','2026-09-15','Revision required before final acceptance. ');
INSERT INTO NOTIFICATION VALUES

```

```
(701,102,1002,'Your paper has been accepted.','2026-09-15','Sent'),
(702,103,1003,'Your paper requires revision.','2026-09-15','Sent');
```

Listing 7.3: INSERT statements for sample data

7.3 Sample Outputs

Figure 7.1 shows the SHOW TABLES command listing the ten tables of conferencedb, followed by the contents of the ROLE, AUTHOR, CONFERENCE and REVIEWER tables.

```
mysql> SHOW TABLES;
```

Tables_in_conferencedb
author
conference
decision
notification
paper
paper_author
review
reviewer
role
user_account

```
10 rows in set (0.046 sec)
```

```
mysql> SELECT * FROM ROLE;
```

Role_ID	Role_Name	Description
1	Admin	Manages conference and system
2	Author	Submits and tracks papers
3	Reviewer	Reviews submitted papers

```
3 rows in set (0.150 sec)
```

```
mysql> SELECT * FROM AUTHOR;
```

Author_ID	Name	Email	Affiliation
101	Rahul Sharma	rahul@email.com	Woxsen University
102	Priya Reddy	priya@email.com	IIT Hyderabad
103	Arjun Kumar	arjun@email.com	Osmania University

```
3 rows in set (0.015 sec)
```

```
mysql> SELECT * FROM CONFERENCE;
```

Conference_ID	Conference_Name	Start_Date	End_Date	Location
1	Woxsen AI Conference 2026	2026-10-20	2026-10-22	Hyderabad

```
1 row in set (0.015 sec)
```

```
mysql> SELECT * FROM REVIEWER;
```

Reviewer_ID	Name	Email	Expertise	Current_Load
201	Dr. Anil Mehta	anil@reviewer.com	Artificial Intelligence	2
202	Dr. Neha Rao	neha@reviewer.com	Machine Learning	1

```
2 rows in set (0.010 sec)
```

Figure 7.1: SHOW TABLES and contents of ROLE, AUTHOR, CONFERENCE and REVIEWER

Figure 7.2 shows the contents of USER_ACCOUNT, PAPER, PAPER_AUTHOR, REVIEW, DECISION and NOTIFICATION.

```
mysql> SELECT * FROM USER_ACCOUNT;
```

User_ID	Username	Password	Role_ID
301	admin01	admin123	1
302	rahul01	rahul123	2
303	priya01	priya123	2
304	anil01	anil123	3

4 rows in set (0.007 sec)

```
mysql> SELECT * FROM PAPER;
```

Paper_ID	Title	Abstract	Submission_Date	Status	Conference_ID
1001	AI in Healthcare	Application of AI in modern healthcare systems	2026-09-10	Under Review	1
1002	Secure ML Systems	Security techniques for machine learning systems	2026-09-11	Accepted	1
1003	NLP for Education	Using NLP to improve educational systems	2026-09-12	Revision	1

3 rows in set (0.007 sec)

```
mysql> SELECT * FROM PAPER_AUTHOR;
```

Paper_ID	Author_ID
1001	101
1001	102
1002	102
1003	103

4 rows in set (0.007 sec)

```
mysql> SELECT * FROM REVIEW;
```

Review_ID	Paper_ID	Reviewer_ID	Score	Comments	Review_Date
501	1001	201	8	Strong methodology and good implementation.	2026-09-13
502	1002	202	9	Excellent research work.	2026-09-14
503	1003	201	6	The paper needs improvement in methodology.	2026-09-14

```
mysql> SELECT * FROM REVIEW;
```

Review_ID	Paper_ID	Reviewer_ID	Score	Comments	Review_Date
501	1001	201	8	Strong methodology and good implementation.	2026-09-13
502	1002	202	9	Excellent research work.	2026-09-14
503	1003	201	6	The paper needs improvement in methodology.	2026-09-14

3 rows in set (0.008 sec)

```
mysql> SELECT * FROM DECISION;
```

Decision_ID	Paper_ID	Outcome	Decision_Date	Remarks
601	1002	Accept	2026-09-15	Paper meets conference requirements.
602	1003	Revise	2026-09-15	Revision required before final acceptance.

2 rows in set (0.007 sec)

```
mysql> SELECT * FROM NOTIFICATION;
```

Notification_ID	Author_ID	Paper_ID	Message	Notification_Date	Status
701	102	1002	Your paper has been accepted.	2026-09-15	Sent
702	103	1003	Your paper requires revision.	2026-09-15	Sent

2 rows in set (0.007 sec)

Figure 7.2: Contents of USER_ACCOUNT, PAPER, PAPER_AUTHOR, REVIEW, DECISION and NOTIFICATION

8. QUERIES WITH OUTPUTS

This chapter lists the SQL queries used to retrieve and analyse the data, each with its output on the sample data.

Query 1: Papers with their conference and status

```
SELECT p.Paper_ID, p.Title, p.Status, c.Conference_Name
FROM PAPER p
JOIN CONFERENCE c ON p.Conference_ID = c.Conference_ID;
```

Paper_ID	Title	Status	Conference_Name
1001	AI in Healthcare	Under Review	Woxsen AI Conference 2026
1002	Secure ML Systems	Accepted	Woxsen AI Conference 2026
1003	NLP for Education	Revision	Woxsen AI Conference 2026

Output 8.1: Papers with conference and status

Query 2: Authors of each paper (many-to-many join)

```
SELECT p.Title, a.Name AS Author
FROM PAPER p
JOIN PAPER_AUTHOR pa ON p.Paper_ID = pa.Paper_ID
JOIN AUTHOR a ON pa.Author_ID = a.Author_ID
ORDER BY p.Paper_ID;
```

Title	Author
AI in Healthcare	Rahul Sharma
AI in Healthcare	Priya Reddy
Secure ML Systems	Priya Reddy
NLP for Education	Arjun Kumar

Output 8.2: Authors of each paper

Query 3: Average review score per paper

```
SELECT p.Title, AVG(r.Score) AS Avg_Score
FROM PAPER p
JOIN REVIEW r ON p.Paper_ID = r.Paper_ID
GROUP BY p.Paper_ID, p.Title;
```

Title	Avg_Score
AI in Healthcare	8.0000
Secure ML Systems	9.0000
NLP for Education	6.0000

Output 8.3: Average score per paper

Query 4: Number of reviews done by each reviewer

```
SELECT rv.Name, COUNT(r.Review_ID) AS Reviews_Done
FROM REVIEWER rv
LEFT JOIN REVIEW r ON rv.Reviewer_ID = r.Reviewer_ID
GROUP BY rv.Reviewer_ID, rv.Name;
```

Name	Reviews_Done
Dr. Anil Mehta	2
Dr. Neha Rao	1

Output 8.4: Reviewer workload

Query 5: Number of decisions by outcome

```
SELECT Outcome, COUNT(*) AS Total
FROM DECISION
GROUP BY Outcome;
```

Outcome	Total
Accept	1
Revise	1

Output 8.5: Decisions by outcome

Query 6: Papers still waiting for a decision

```
SELECT Title
FROM PAPER
WHERE Paper_ID NOT IN (SELECT Paper_ID FROM DECISION);
```

Title
AI in Healthcare

Output 8.6: Papers without a decision

Query 7: Acceptance rate

```
SELECT ROUND(100 * SUM(Outcome = 'Accept') / COUNT(*), 2)
      AS Acceptance_Rate
FROM DECISION;
```

Acceptance_Rate
50.00

Output 8.7: Acceptance rate (%)

Query 8: Notifications sent to authors

```
SELECT a.Name, n.Message, n.Status
FROM NOTIFICATION n
JOIN AUTHOR a ON n.Author_ID = a.Author_ID;
```

Name	Message	Status
Priya Reddy	Your paper has been accepted.	Sent
Arjun Kumar	Your paper requires revision.	Sent

Output 8.8: Notifications

Query 9: Query given during Presentation-II

```
-- [Paste the exact query shown in Presentation-II here]
```

[Column 1]	[Column 2]	[Column 3]
[output]	[output]	[output]

Output 8.9: Presentation-II query

9. UI DESIGN AND SCREENSHOTS

The user interface gives each role a view of the same database. Access is decided by the Role_ID of the logged-in USER_ACCOUNT: administrators manage the conference, authors submit and track papers, and reviewers evaluate assigned papers. The planned screens are described below; insert the screenshots of your application in the marked places.

9.1 Login Screen

Users sign in with their username and password. The system looks up the user's role and opens the matching dashboard.

9.2 Admin Dashboard

Administrators manage conferences, view all papers and statuses, assign reviewers, and see statistics such as submissions, review progress and acceptance rate.

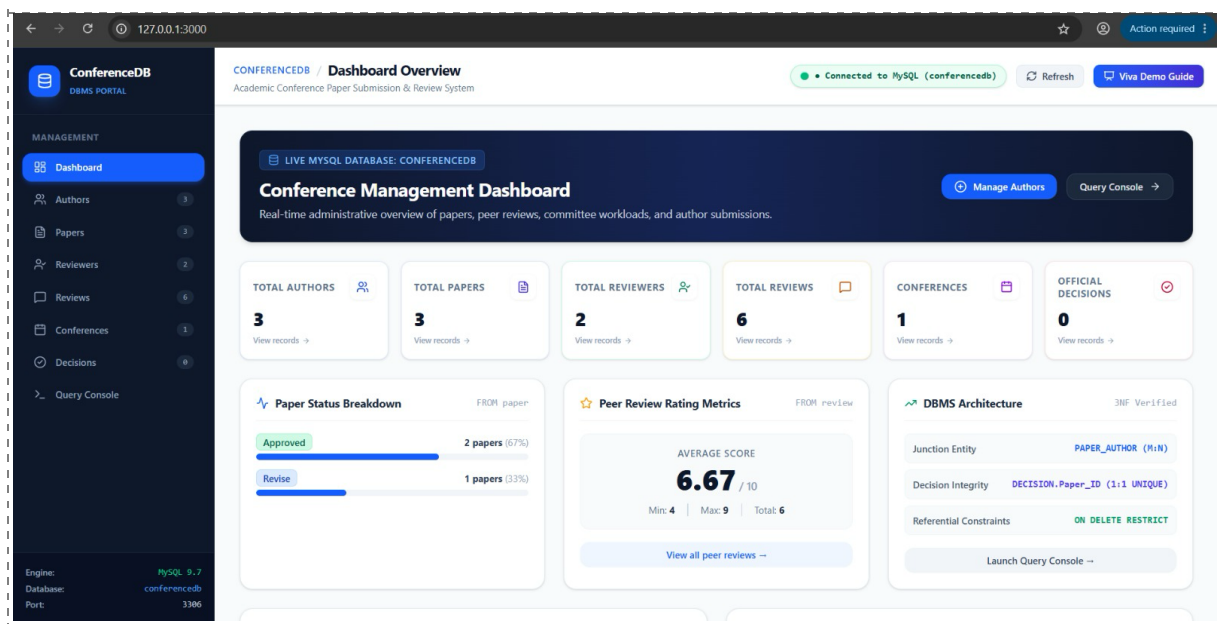


Figure 9.2: Admin Dashboard

9.3 Author Paper Submission and Tracking

Authors submit a paper with title, abstract and co-authors, and track its status (Under Review, Accepted, Revision) and notifications.

ConferenceDB
DBMS PORTAL

MANAGEMENT

- Dashboard
- Authors
- Papers
- Reviewers
- Reviews**
- Conferences
- Decisions
- Query Console

Engine: MySQL 9.7
Database: conferencedb
Port: 3306

CONFERENCEDB / Peer Reviews
Academic Conference Paper Submission & Review System

Connected to MySQL (conferencedb) Refresh Viva Demo Guide

Reviews
Table: review + Joined with paper, reviewer

Search reviews... + Add Review

REVIEW ID	PAPER TITLE	REVIEWER	SCORE	COMMENTS	DATE	ACTIONS
#504	AI in Healthcare	Dr. Anil Mehta	8 / 10	"Good methodology and implementation."	2026-09-21	View Edit
#505	AI in Healthcare	Dr. Neha Rao	6 / 10	"Needs improvement in experimental results."	2026-09-21	View Edit
#506	Secure ML Systems	Dr. Anil Mehta	9 / 10	"Excellent research and strong results."	2026-09-21	View Edit
#507	Secure ML Systems	Dr. Neha Rao	8 / 10	"Well structured and technically sound."	2026-09-21	View Edit
#508	NLP for Education	Dr. Anil Mehta	4 / 10	"Major improvements required."	2026-09-21	View Edit
#509	NLP for Education	Dr. Neha Rao	5 / 10	"Results are not sufficiently convincing."	2026-09-21	View Edit

Figure 9.3: Author Submission and Tracking Screen

9.4 Reviewer Panel

Reviewers see their assigned papers and submit a score and comments.

ConferenceDB
DBMS PORTAL

MANAGEMENT

- Dashboard
- Authors
- Papers
- Reviewers**
- Reviews
- Conferences
- Decisions
- Query Console

Engine: MySQL 9.7
Database: conferencedb
Port: 3306

CONFERENCEDB / Reviewer Committee
Academic Conference Paper Submission & Review System

Connected to MySQL (conferencedb) Refresh Viva Demo Guide

Reviewers
Table: reviewer + Primary Key: Reviewer_ID

Search reviewers... + Add Reviewer

REVIEWER ID	NAME	EMAIL	EXPERTISE	CURRENT LOAD	REVIEWS DONE	ACTIONS
#201	Dr. Anil Mehta	anil@reviewer.com	Artificial Intelligence	2	3 completed	View Edit
#202	Dr. Neha Rao	neha@reviewer.com	Machine Learning	1	3 completed	View Edit

Figure 9.4: Reviewer Panel

10. IMPLEMENTATION DETAILS

10.1 Technology Stack

Layer	Technology
Database	MySQL, database conferencedb, 10 tables
Query language	SQL (DDL and DML)
Front end	[Technology used]
Back end	[Technology used]
Version control	Git, GitHub

Table 10.1: Technology Stack

10.2 Database Connectivity

The application layer connects to MySQL using the database name conferencedb, a database user and a password, and sends SQL statements through a connector library. [Add the connection code of your application here, with credentials removed.]

10.3 Key Code Snippets

The following queries are central to the application. The first authenticates a user and finds the role that decides access:

```
SELECT u.User_ID, u.Username, r.Role_Name
FROM USER_ACCOUNT u
JOIN ROLE r ON u.Role_ID = r.Role_ID
WHERE u.Username = 'rahul01' AND u.Password = 'rahul123';
```

The second records a decision and updates the paper status:

```
INSERT INTO DECISION
VALUES (601, 1002, 'Accept', '2026-09-15',
      'Paper meets conference requirements.');
```

```
UPDATE PAPER SET Status = 'Accepted' WHERE Paper_ID = 1002;
```

The third lists the average score of each paper for the programme committee:

```
SELECT p.Title, AVG(r.Score) AS Avg_Score
FROM PAPER p
JOIN REVIEW r ON p.Paper_ID = r.Paper_ID
GROUP BY p.Paper_ID, p.Title;
```

10.4 Integrity and Access Control

- **Referential integrity:** PRIMARY KEY and FOREIGN KEY constraints keep every paper, review and decision linked to valid parents.
- **Validation:** UNIQUE on Email, Username and DECISION.Paper_ID prevents duplicates; NOT NULL protects mandatory fields.
- **Role-based access:** The ROLE table (Admin, Author, Reviewer) separates what each user can do.

11. TESTING

Testing covered structure creation, data loading, query results and constraint enforcement. The constraint tests were executed on a MySQL-compatible server using the schema in Chapter 7, and the error codes below were returned by the server.

11.1 Test Cases and Results

ID	Test case	Expected result	Actual result	Status
T1	Run SHOW TABLES on conferencedb	10 tables listed	10 rows in set	Pass
T2	Insert sample data in all tables	Rows accepted (3, 3, 1, 2, 4, 3, 4, 3, 2, 2)	All rows inserted and displayed	Pass
T3	Join queries (Queries 1–8)	Correct joined and aggregated data	Outputs match the stored data	Pass
T4	Insert an author with an existing e-mail	Rejected by UNIQUE	ERROR 1062 (23000): Duplicate entry	Pass
T5	Insert a paper with a non-existent Conference_ID	Rejected by FOREIGN KEY	ERROR 1452 (23000): foreign key constraint fails	Pass
T6	Insert a second decision for the same paper	Rejected by UNIQUE	ERROR 1062 (23000): Duplicate entry	Pass
T7	Insert a user account with a NULL password	Rejected by NOT NULL	ERROR 1048 (23000): Column cannot be null	Pass
T8	Delete a conference that has papers	Blocked by FOREIGN KEY	ERROR 1451 (23000): foreign key constraint fails	Pass
T9	Insert a duplicate (Paper_ID, Author_ID) pair	Rejected by composite PRIMARY KEY	ERROR 1062 (23000): Duplicate entry	Pass
T10	Insert a user account with an existing username	Rejected by UNIQUE	ERROR 1062 (23000): Duplicate entry	Pass

Table 11.1: Test Cases and Results

11.2 Observations

All tests behaved as designed. The database correctly rejects invalid data and keeps related tables consistent, so errors are caught at the database level and not only in the application.

12. CONCLUSION AND FUTURE ENHANCEMENTS

12.1 Conclusion

This project designed and implemented a relational database for academic conference management. The ER model was converted into ten normalised tables with proper keys and constraints, loaded with sample data and tested with analytical queries and constraint checks. The system centralises submissions, supports a structured review workflow, records decisions and notifications, and separates users by role. It reduces manual processing, improves transparency and can grow with the number of papers and users.

12.2 Future Enhancements

- **Password security:** Store passwords as salted hashes (for example bcrypt) instead of plain text; the sample data currently holds plain-text passwords for demonstration.
- **Account linking:** Add Author_ID and Reviewer_ID foreign keys to USER_ACCOUNT so each login maps directly to an author or reviewer record.
- **Automatic reviewer assignment:** Suggest reviewers automatically using Expertise and Current_Load, and update Current_Load with triggers.
- **Workflow automation:** Use triggers or stored procedures to set PAPER.Status and create notifications when a decision is recorded.
- **Richer review process:** Support several reviewers per paper with conflict-of-interest checks, multiple review rounds, tracks and sessions.
- **Operations:** Send e-mail alerts, store uploaded paper files, add indexes for large data volumes and schedule regular backups.

13. REFERENCES

- [1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.
- [2] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill, 2019.
- [3] MySQL 8.0 Reference Manual, Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [4] E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377-387, 1970.

- [5] P. P. Chen, "The entity-relationship model: Toward a unified view of data,"
ACM Transactions on Database Systems, vol. 1, no. 1, pp. 9-36, 1976.

APPENDIX: GITHUB REPOSITORY LINK

The complete SQL scripts, application code and documentation are available at:

[GitHub repository:-

<https://github.com/novakdjo45/Academic-Conference-Management>]